

QUIZZIFY: AI-POWERED QUIZ MANAGEMENT SYSTEM

21CSC101T – OBJECT ORIENTED DESIGN AND PROGRAMMING

Submitted by

Sautrik Roy [RA2511003010052]

Shivaani M [RA2511003010066]

Ashwin Kumar N [RA2511003010033]

Ayush Agarwal [RA2511003010012]

Class and Section: A1 | 21CSC101T

under the guidance of

Dr. Vetriselvi D

Assistant Professor

Dept. of CTECH

in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING



**DEPARTMENT OF COMPUTING TECHNOLOGIES
COLLEGE OF ENGINEERING AND TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203**

APRIL 2026

BONAFIDE CERTIFICATE

Certified that the **Object-Oriented Design and Programming using C++ (OODP)** Project titled **“QUIZZIFY: AI-POWERED QUIZ MANAGEMENT SYSTEM”** is the bonafide work of **SAUTRIK ROY [RA2511003010052]**, **SHIVAANI M [RA2511003010066]**, **ASHWIN KUMAR N [RA2511003010033]** and **AYUSH AGARWAL [RA2511003010012]**, who carried out the project work under my guidance. Certified further that, to the best of my knowledge, the students have carried out this project with sincerity and good effort, and the work presented reflects their understanding and learning in the subject.

Dr. VETRISSELVI D

Assistant Professor

Dept. of CTECH, SRMIST.

Dr. NIRANJANA G

Professor and Head

Dept. of CTECH, SRMIST.

INDEX

1. Introduction	4
2. Implementation of Design Concepts	5
2.1 Encapsulation in Data Security	5
2.2 Abstraction of Complex Workflows	6
2.3 Inheritance for Modular Test Modes	6
2.4 Polymorphism in Execution Logic	6
3. UML Diagrams	7
3.1 Use Case Diagram	7
3.2 System Architecture	7
3.3 Sequence Diagram	8
3.4 Class Diagram	8
3.5 Activity Diagram	9
4. Application Interfaces	11
4.1 Platform Landing Page	11
4.2 System Input: Document Forging	12
4.3 System Input: Active Assessment Interface	13
4.4 System Output: Assessment Results	14
4.5 System Output: Global Leaderboard	15
5. Coding Implementation	16
5.1 Backend Core Engine (C++)	16
5.2 Frontend Code (JavaScript)	22
6. Conclusion	23
6.1 Future Scope	24
7. References	24

1. Introduction

The Quizzify Quiz Management System represents a robust and scalable approach to educational technology, bridging the gap between automated content generation and structured software engineering. At its core, the application integrates modern full-stack web technologies with Google's Gemini Flash AI model. This intelligent pipeline parses complex user-uploaded study materials—such as PDFs and PowerPoints—and seamlessly converts them into structured, competitive assessments. The platform aims to reduce the manual overhead of traditional quiz creation while simultaneously offering students an adaptive, highly engaging learning experience.

Throughout the development lifecycle, the architectural design was strictly guided by core software engineering methodologies, specifically Object-Oriented Design Principles (OODP). By modeling the system's domains through objects, classes, and defined interactions, the application achieves a high degree of modularity, memory safety, and maintainability. The backend logic, conceptually structured in C++, handles the complex rules of the assessment engine, while the frontend React application seamlessly renders the user interface. This separation of concerns ensures that the application can effortlessly scale to accommodate diverse testing modes and complex user interactions.

2. Implementation of Design Concepts

The fundamental architecture of Quizzify is deeply rooted in Object-Oriented Design. Rather than executing as a procedural script, the system instantiates discrete objects that encapsulate their own state and behavior. The following outlines precisely how and where these core principles were utilized within the platform to drive functionality and security.

2.1 Encapsulation in Data Security

Encapsulation ensures the integrity of correct answers and the user's running score. Within the system, objects such as the `Question` class encapsulate these attributes as private members. The React frontend interfaces with this data solely through controlled API endpoints. When a user selects an answer, the system does not expose the correct answer key to the browser; instead, it sends the selection to an encapsulated algorithm, ensuring malicious actors cannot tamper with the score.

2.2 Abstraction of Complex Workflows

Abstraction simplifies the interaction between core logic and external services. The process of connecting to the Google Gemini AI, formatting text, sending the API prompt, and parsing JSON involves substantial complexity. This workflow is abstracted behind a clean generation route. When a user clicks "Forge Quiz", they are completely shielded from the underlying algorithms, interacting only with the high-level abstraction.

2.3 Inheritance for Modular Test Modes

Inheritance was implemented to reduce code duplication and promote modular scalability. The application supports different testing environments, such as Practice Mode and Competitive Mode. A base **Assessment** class contains shared attributes like the question array and total score. Derived classes like **PracticeQuiz** and **CompetitiveTest** inherit this logic but introduce their own specialized rules, such as countdown timers.

2.4 Polymorphism in Execution Logic

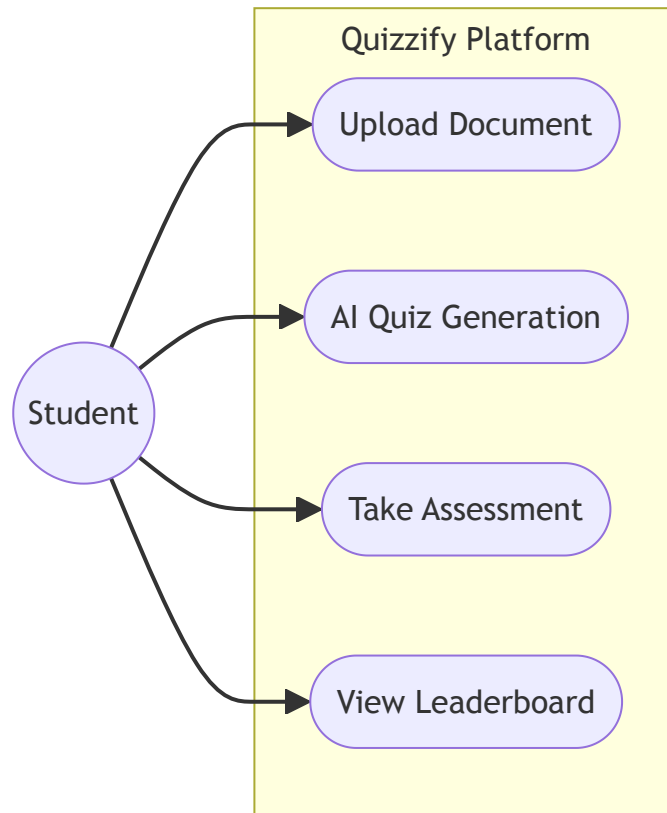
Polymorphism allows the system to dynamically execute logic based on the assessment type. Using overridden virtual functions, the system triggers the `conductAssessment()` method identically across all modes. However, the runtime behavior changes polymorphically. In a Competitive Test, it strictly enforces time limits; in a Practice Quiz, it reveals correct answers instantly. This ensures the primary execution loop remains clean and decoupled.

3. UML Diagrams

The system's architecture is modeled using simplified Unified Modeling Language diagrams to clearly illustrate object interactions and execution flow. UML provides a standardized visual language that helps bridge the gap between abstract design and concrete software implementation. By visualizing the structural hierarchy and behavioral workflows, these diagrams explicitly demonstrate how Object-Oriented principles are enforced across the modules, ensuring a shared understanding of the system's security and automated pipelines before execution.

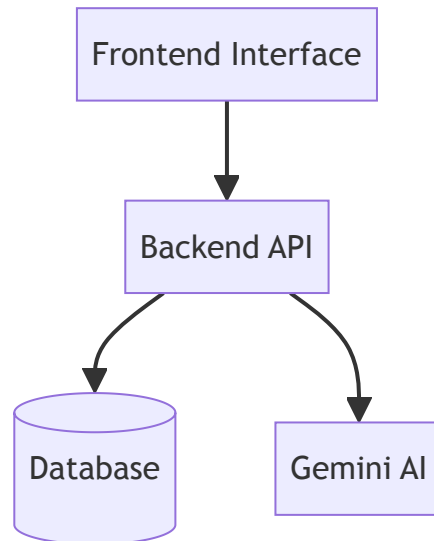
3.1 Use Case Diagram

The Use Case diagram illustrates the basic functionalities available to a Student using the platform, from uploading documents to viewing the global leaderboard.



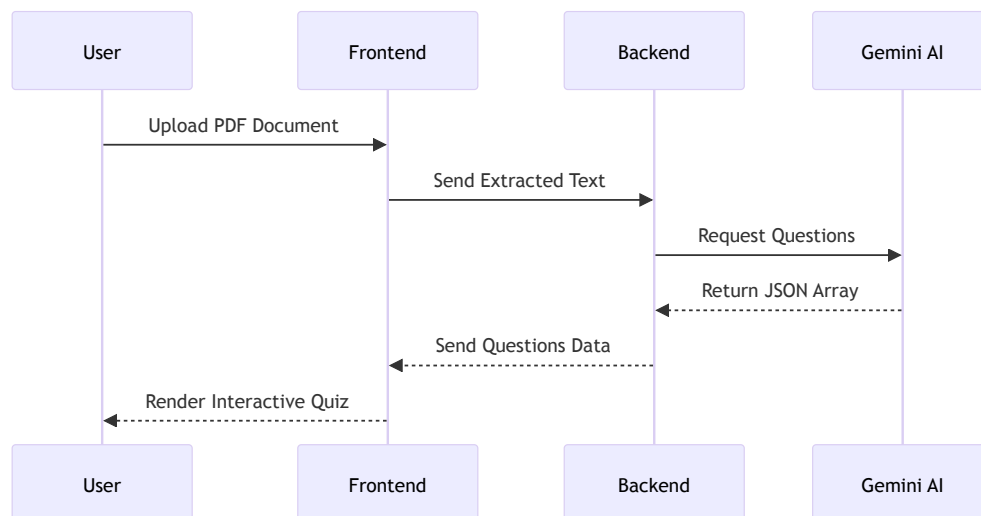
3.2 System Architecture

This diagram shows the high-level flow of data from the frontend interface to the backend database and AI model.



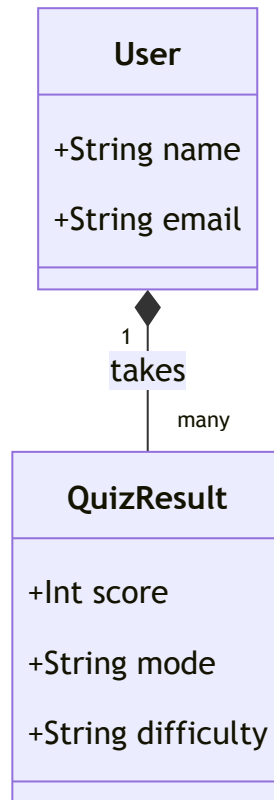
3.3 Sequence Diagram

The Sequence Diagram models the simple chronological steps taken when a user generates a quiz using AI.



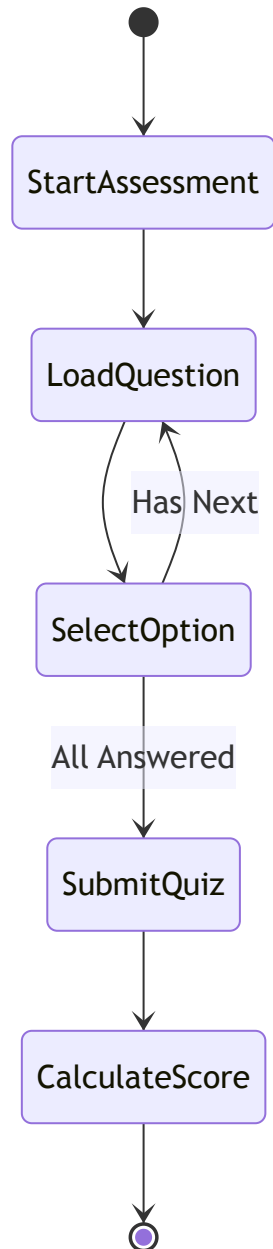
3.4 Class Diagram

The Class Diagram visually represents the two core entities in the system: the User and the Quiz Result.



3.5 Activity Diagram

The Activity Diagram maps the straightforward execution flow of a user attempting a multiple-choice quiz.

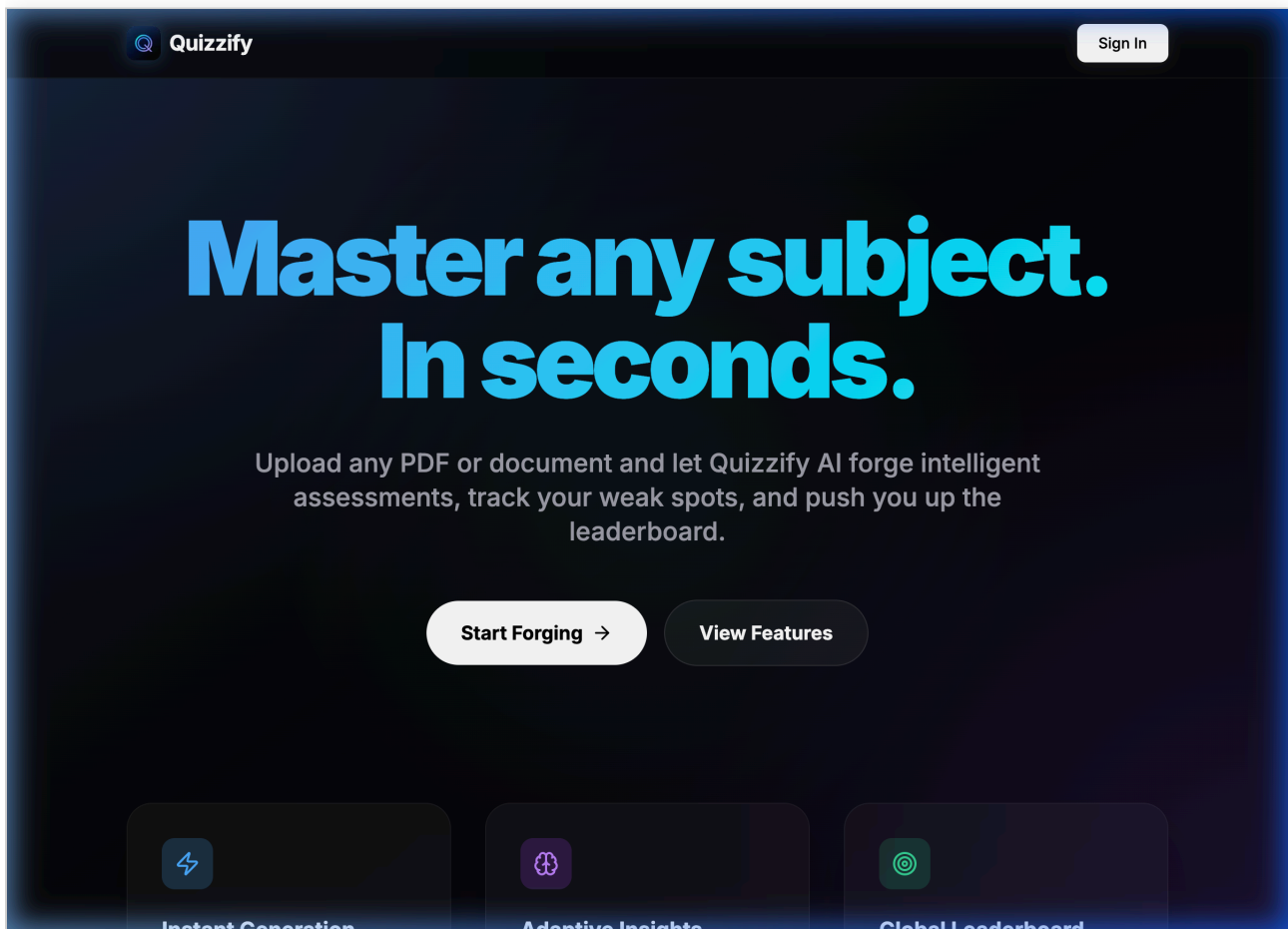


4. Application Interfaces

The following screenshots demonstrate how the previously described Object-Oriented concepts manifest practically within the application's user interface, detailing both the inputs required from the user and the outputs generated by the system algorithms.

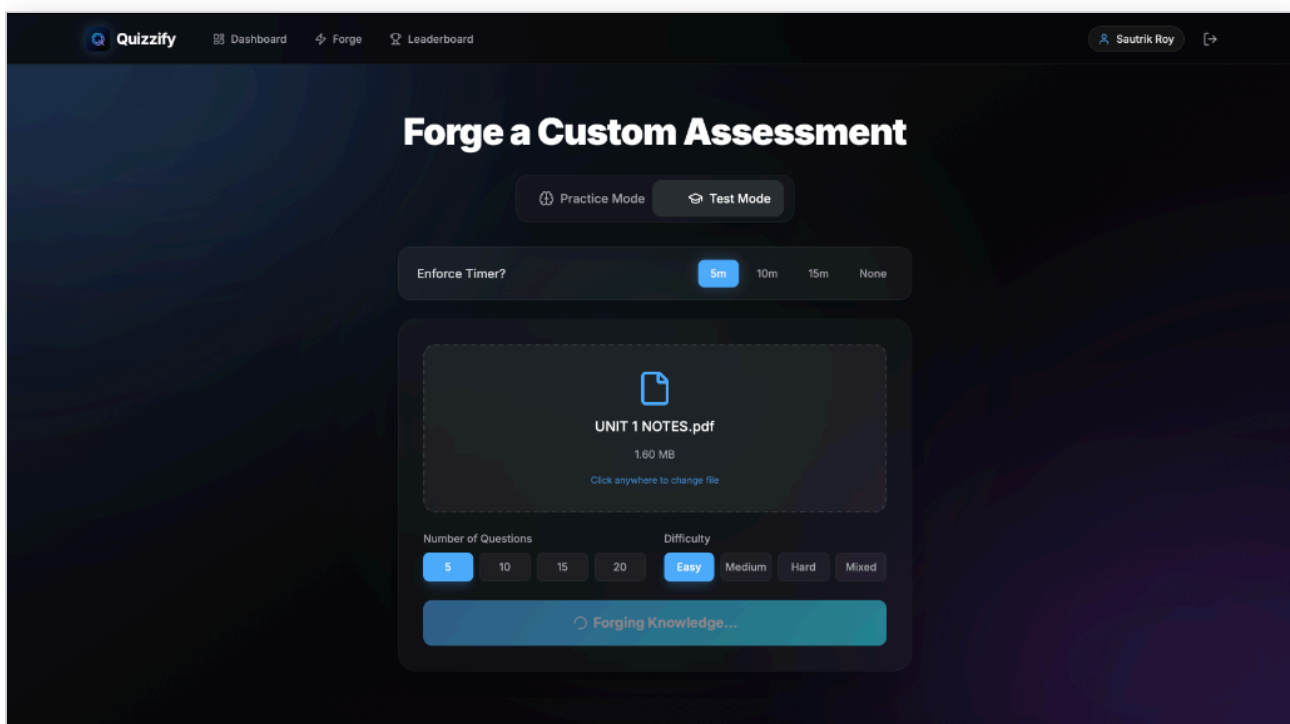
4.1 Platform Landing Page

The landing page serves as the entry point to the application. At this stage, the system abstracts the complex backend processes. The user is presented with a clean interface that introduces the platform's capabilities before they authenticate and initiate the creation of User and Session objects within the system.



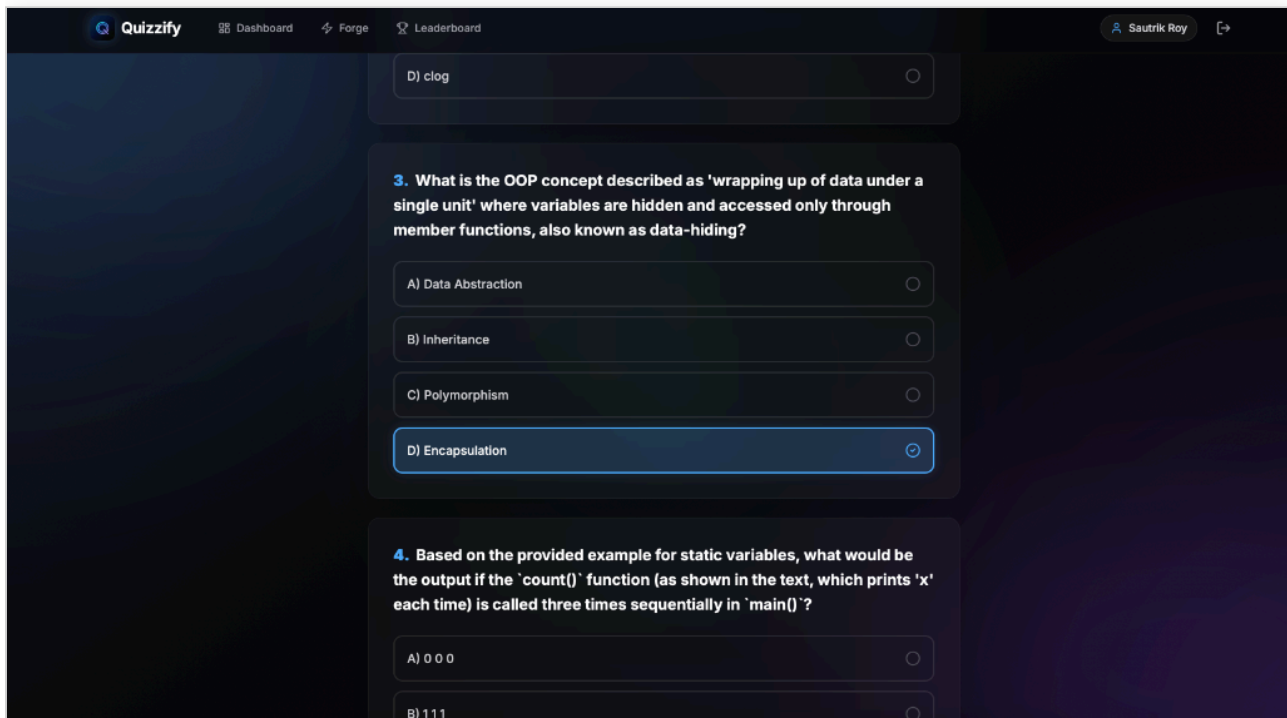
4.2 System Input: Document Forging

This interface represents the initial document upload phase where Abstraction is heavily utilized. The user simply uploads their notes (e.g., "UNIT 1 NOTES.pdf") and selects the difficulty and question count. The complex underlying process of parsing the PDF and querying the Google Gemini AI is entirely abstracted away from the user, providing a clean, one-click experience.



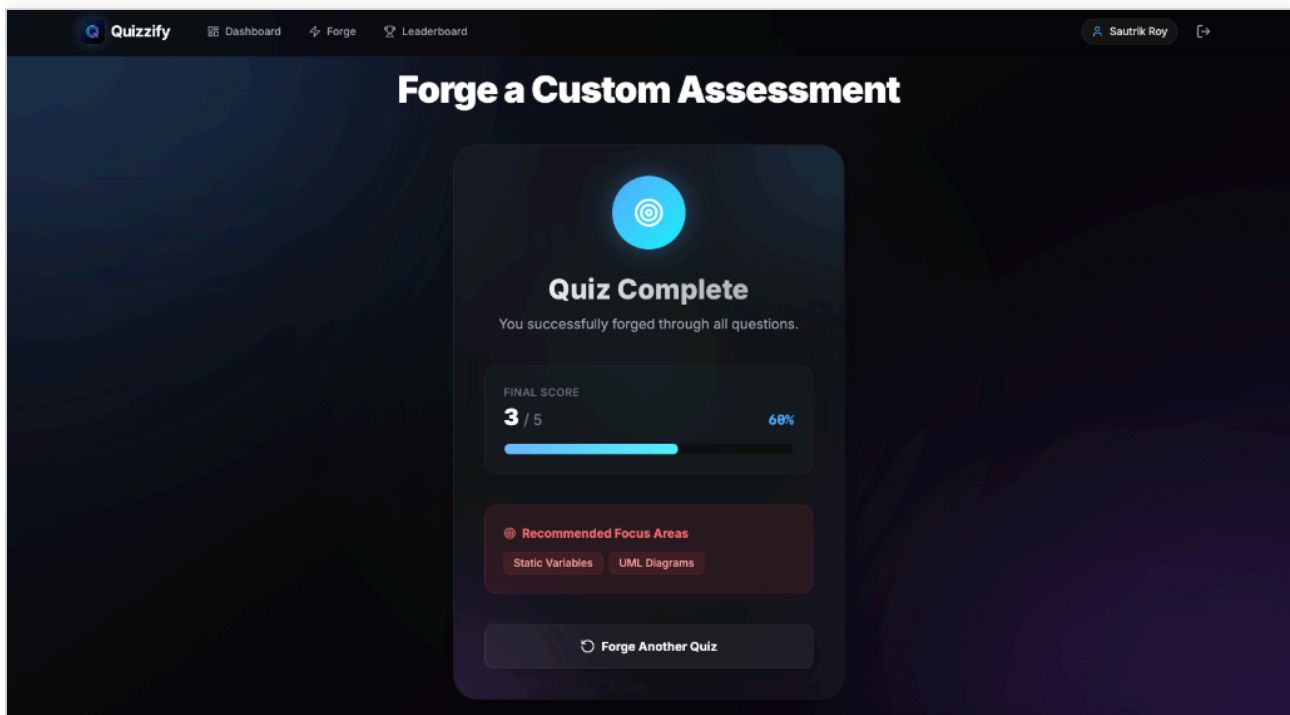
4.3 System Input: Active Assessment Interface

This interface acts as the primary input mechanism where the user actively participates in the assessment. This view heavily relies on Encapsulation. The `Question` objects are rendered securely on the screen, allowing the user to select options without exposing the underlying correct answer key. The state of the active quiz—including the timer and the user's selections—is meticulously managed and protected from external manipulation until the final submission triggers the evaluation methods.



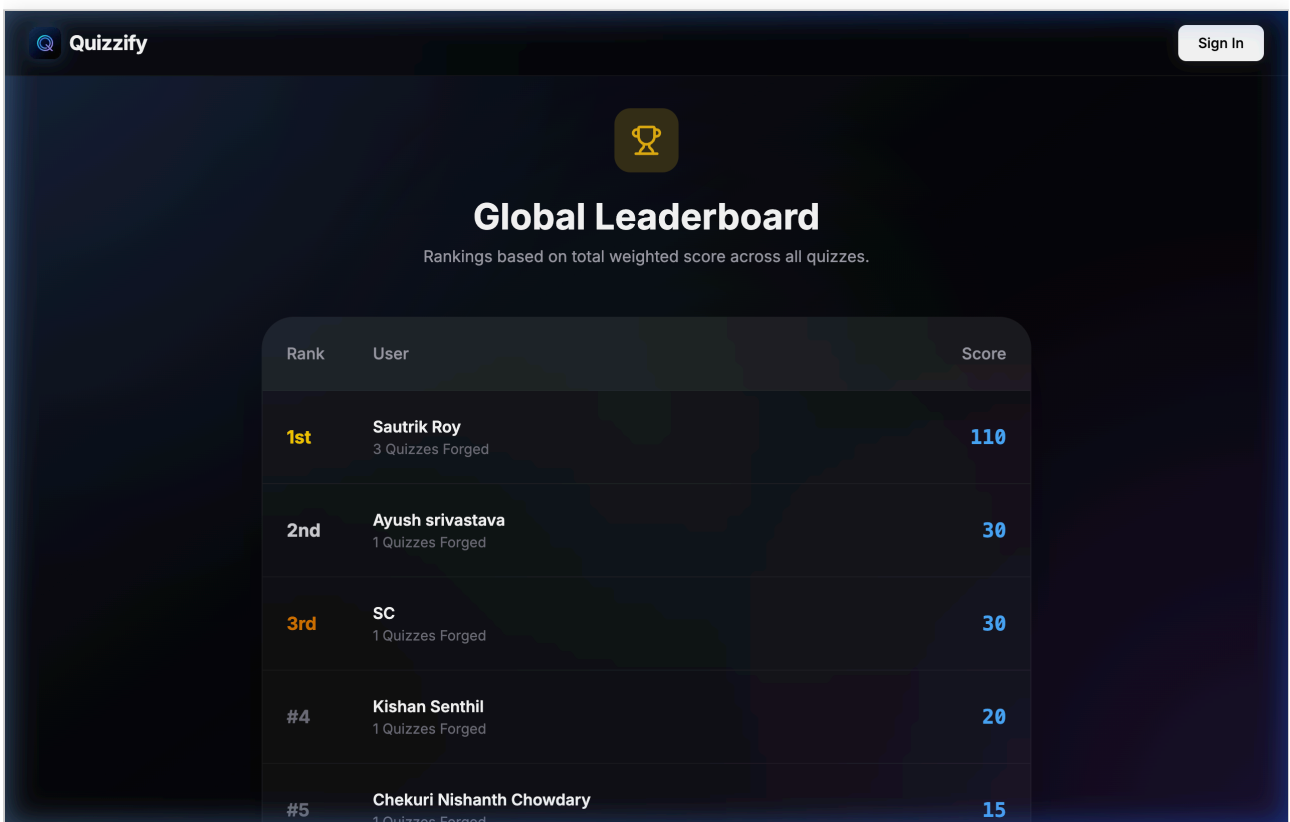
4.4 System Output: Assessment Results

Upon submission, the system triggers its polymorphic scoring algorithms. Depending on whether the user selected a practice or competitive mode, the `evaluateScore()` methods compute the final grade and derive analytical insights. This output screen displays the instantiated `QuizResult` object, detailing the user's score and dynamically highlighting their weak conceptual topics based on incorrect inputs.



4.5 System Output: Global Leaderboard

The final output of the application is the global leaderboard. Once the assessment is complete, the encapsulated score data is transmitted to the Next.js API and securely persisted in the PostgreSQL database. This interface queries the database to rank and display the top-performing User objects, dynamically updating in real-time as new assessments are completed.



Rank	User	Score
1st	Sautrik Roy 3 Quizzes Forged	110
2nd	Ayush srivastava 1 Quizzes Forged	30
3rd	SC 1 Quizzes Forged	30
#4	Kishan Senthil 1 Quizzes Forged	20
#5	Chekuri Nishanth Chowdary 1 Quizzes Forged	15

5. Coding Implementation

The implementation of the system relies on a combination of a structured C++ backend engine to model the core domain logic, and a modern JavaScript API to handle web communications and integrations.

5.1 Backend Core Engine (C++)

This codebase represents the pure application of the Object-Oriented concepts. It defines the abstract base classes, enforces data protection through private access modifiers, and implements the polymorphic functions that handle the assessment rules.

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

// Encapsulation – private data, controlled access via public methods
class Question {
private:
    string text;
    vector<string> options;
    int correctOptionIndex;

public:
    // Constructor – initialises fields when object is created
    Question(string q, vector<string> opts, int correctIndex)
        : text(q), options(opts), correctOptionIndex(correctIndex) {}

    // Destructor – called when object goes out of scope
    ~Question() {}

    string getText() const { return text; }
    vector<string> getOptions() const { return options; }
    bool checkAnswer(int selectedIndex) const { return selectedIndex
== correctOptionIndex; }
};

// Abstraction – pure virtual function forces derived classes to
define their own conductAssessment()
// Polymorphism – virtual keyword enables the correct derived version
```



```

to be called at runtime
class Assessment {
protected:
    string title;
    vector<Question> questions;
    int currentScore;

public:
    // Constructor – sets shared state for all derived classes
    Assessment(string t) : title(t), currentScore(0) {}

    // Destructor – virtual so the correct derived destructor also
    runs when deleting via base pointer
    virtual ~Assessment() {}

    void addQuestion(const Question& q) { questions.push_back(q); }

    // Pure virtual – makes Assessment abstract; also the source of
    runtime polymorphism
    virtual void conductAssessment() = 0;

    int getScore() const { return currentScore; }
    string getTitle() const { return title; }
};

// Inheritance – gets title, questions, currentScore from Assessment
// Runtime Polymorphism – overrides conductAssessment() with Practice
behaviour (no timer, instant feedback)
class PracticeQuiz : public Assessment {
public:
    PracticeQuiz(string t) : Assessment(t) {}

    void conductAssessment() override {
        cout << "--- PRACTICE MODE: " << title << " ---" << endl;
        cout << "Hints enabled. Answers shown immediately after each
question." << endl;
        currentScore = questions.size() * 10;
        cout << "Practice complete! Score: " << currentScore << endl;
    }
};

```

```

// Inheritance – gets title, questions, currentScore from Assessment
// Runtime Polymorphism – overrides conductAssessment() with Test
behaviour (timed, score saved to leaderboard)
class CompetitiveTest : public Assessment {
private:
    int timeLimitSeconds;

public:
    CompetitiveTest(string t, int timeLimit) : Assessment(t),
timeLimitSeconds(timeLimit) {}

    void conductAssessment() override {
        cout << "*** TEST MODE: " << title << " ***" << endl;
        cout << "Time Limit: " << timeLimitSeconds << " seconds. No
hints." << endl;
        currentScore = questions.size() * 10;
        cout << "Exam complete! Score saved to leaderboard: " <<
currentScore << endl;
    }
};

// Inheritance base – shared interface for Student and Admin
class User {
protected:
    string name;
    string email;

public:
    User(string n, string e) : name(n), email(e) {}
    virtual void displayDashboard() const { cout << "User: " << name
<< endl; }
    string getName() const { return name; }
    virtual ~User() {}
};

// Inheritance – gets name, email from User
// Runtime Polymorphism – overrides displayDashboard() with student-
specific stats
class Student : public User {

```

```

private:
    int totalScore;
    int quizzesTaken;

public:
    Student(string n, string e) : User(n, e), totalScore(0),
    quizzesTaken(0) {}

    void recordAssessment(Assessment* assessment) {
        assessment->conductAssessment();
        totalScore += assessment->getScore();
        quizzesTaken++;
    }

    void displayDashboard() const override {
        cout << "===== " << endl;
        cout << "STUDENT DASHBOARD: " << name << endl;
        cout << "Total Score: " << totalScore << endl;
        cout << "Quizzes Taken: " << quizzesTaken << endl;
        cout << "===== " << endl;
    }
};

// Inheritance – gets name, email from User
// Runtime Polymorphism – overrides displayDashboard() with admin-
specific view
class Admin : public User {
public:
    Admin(string n, string e) : User(n, e) {}

    void displayDashboard() const override {
        cout << "ADMIN DASHBOARD: " << name << endl;
        cout << "System Status: Online. Leaderboard: Synchronized." <<
endl;
    }
};

int main() {
    cout << "\n=== QUIZZIFY AI – C++ CORE ENGINE ===\n" << endl;

```

```

    // Constructor fires here – Question objects created and fields
    initialised
    Question q1("What is a kernel?", {"Hardware", "Core OS component",
    "App layer"}, 1);
    Question q2("What is deadlock?", {"Process starvation", "Memory
    error", "Endless wait"}, 2);

    // Polymorphism – base class pointer holds derived objects;
    correct version called at runtime
    Assessment* practice = new PracticeQuiz("OS Fundamentals –
    Practice");
    Assessment* exam      = new CompetitiveTest("OS Fundamentals –
    Ranked Exam", 600);

    practice->addQuestion(q1);
    exam->addQuestion(q2);

    Student student("Krish", "krish@university.edu");
    Admin    sysAdmin("System Overseer", "root@quizzify.com");

    cout << "\n--- Practice Mode ---" << endl;
    student.recordAssessment(practice);

    cout << "\n--- Test Mode ---" << endl;
    student.recordAssessment(exam);

    // Runtime Polymorphism – same call on different User types,
    different output
    cout << "\n--- Dashboards ---" << endl;
    User* users[] = { &student, &sysAdmin };
    for (const auto* u : users) {
        u->displayDashboard();
        cout << endl;
    }

    // Destructor fires here – virtual destructor ensures correct
    cleanup
    delete practice;
    delete exam;

    cout << "\n=== SYSTEM SHUTDOWN COMPLETE ===" << endl;

```

```
    return 0;
}
```

5.2 Frontend Code (JavaScript)

This TypeScript code illustrates the intelligent API route responsible for abstracting the communication with the Google Gemini AI. It securely formats the prompt, handles asynchronous requests, and ensures the data returned strictly adheres to the expected JSON structure before it is parsed by the frontend components.

```
import { NextResponse } from "next/server";
import { GoogleGenerativeAI } from "@google/generative-ai";
import { getServerSession } from "next-auth";
import { authOptions } from "@/lib/nextauth";

export async function POST(req: Request) {
  try {
    const session = await getServerSession(authOptions);
    if (!session || !session.user) {
      return NextResponse.json({ error: "Unauthorized. Please log in to forge quizzes." }, { status: 401 });
    }

    const formData = await req.formData();
    const extractedText = formData.get("text") as string | null;
    const count = parseInt(formData.get("count") as string || "5", 10);
    const difficulty = formData.get("difficulty") as string || "medium";

    if (!extractedText || !extractedText.trim()) {
      return NextResponse.json({ error: "Empty or invalid document text." }, { status: 400 });
    }

    const apiKey = process.env.GEMINI_API_KEY;
    if (!apiKey) throw new Error("GEMINI_API_KEY environment variable is missing.");
  }
}
```

```

const genAI = new GoogleGenerativeAI(apiKey);
const model = genAI.getGenerativeModel({ model: "gemini-2.5-flash"
});

const difficultyInstruction = difficulty === "mixed"
  ? "a MIXED difficulty level (e.g., approximately 30% easy, 40%
medium, 30% hard)"
  : `a ${difficulty.toUpperCase()} difficulty level`;

const prompt = `You are an expert test creator.
Extract key concepts from the following text and generate exactly
${count} multiple-choice questions at ${difficultyInstruction}.

Output strictly as a raw JSON array of objects fulfilling this
structure:
[
  {
    "question": "The question string",
    "options": ["A", "B", "C", "D"],
    "answer": "The exact string of the correct option",
    "topic": "A 1-3 word classification of the concept tested"
  }
]

Do not include any formatting, markdown wrappers, or extra text. ONLY
return the JSON array.

--- Source Text ---
${extractedText.slice(0, 30000)}`;

const result = await model.generateContent({
  contents: [{ role: "user", parts: [{ text: prompt }] }]
});
const responseText = result.response.text();
const cleanedText = responseText.replace(/```json/g,
""").replace(/```/g, "").trim();
const questions = JSON.parse(cleanedText);

return NextResponse.json({ questions });
} catch (error: any) {
  console.error("API Pipeline Error:", error);

```

```
    return NextResponse.json(  
      { error: error.message || "Failed to generate questions" },  
      { status: 500 }  
    );  
  }  
}
```

6. Conclusion

The development of the Quizzify platform successfully demonstrates the integration of advanced artificial intelligence within a rigorously structured software architecture. By anchoring the application design in Object-Oriented Design Principles—Encapsulation, Abstraction, Inheritance, and Polymorphism—the system achieved a high degree of security, maintainability, and modularity.

The strict encapsulation of state logic ensures that the application maintains perfect academic integrity during active assessments. The abstraction of the complex AI generation pipeline simplifies the user experience, allowing educators and students to generate dynamic quizzes seamlessly. Furthermore, the use of inheritance and polymorphism guarantees that the platform can effortlessly scale to accommodate new assessment methodologies in the future without requiring extensive rewrites of the foundational code.

Ultimately, Quizzify proves that combining structured software design with modern web frameworks results in a production-ready application capable of drastically transforming the educational landscape, reducing manual workloads, and deeply engaging students through adaptive, competitive learning environments.

6.1 Future Scope

Looking ahead, the robust Object-Oriented foundation of Quizzify allows for seamless integration of advanced features. Future enhancements will focus on extending the Abstraction layer to support multimedia study materials, such as parsing educational video transcripts or audio lectures. Additionally, the inherited **Assessment** models will be expanded to include collaborative multiplayer quiz modes, further leveraging Polymorphism to introduce real-time competitive scoring without disrupting the existing single-player architecture.

7. References

- Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley Professional.
 - Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
 - React Documentation. (2025). Meta Platforms, Inc. Available at: <https://react.dev/>
 - Google DeepMind. (2025). Gemini AI API Documentation. Available at: <https://ai.google.dev/>
-